UFR de mathématique et d'informatique
Université de Strasbourg

LLM-Based Battery Datasheet

Information Extraction and Validation

Ashkan Motamedifar ▪ **Kian Abdolmohammadian**

May 2026

Instructor: Assoc. Prof. Ahmed Samet
Université de Strasbourg

Outline

- Problem & Motivation
- Pipeline Architecture
- Corpus
- Level 1 Extraction
- Level 2 Knowledge Graph
- Level 3 Validation
- Conclusion

Problem & Motivation

The shared challenge: PDFs vary by vendor, units are inconsistent, and LLMs can hallucinate values that are not in the document.

The question is not can we extract it is can we trust the output.

What we chose to do differently:

The Model

Claude Opus 4.6

Reads the full PDF directly. No OCR. No pre-processing.

Tables, text, layout all at once.

Key Insight

The _raw field

Every number stores its exact source phrase.

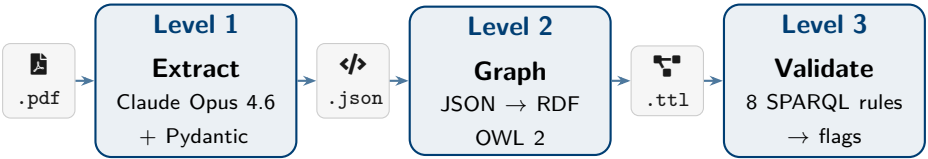
This makes hallucinations detectable.

Pipeline

Three levels

Extract → Graph → Validate. Each level independently testable.

Pipeline Architecture



Each level can be tested alone. **The LLM is no longer a black box — we can check its work.**

Corpus

Provided by the instructor. Four formats, three chemistries — the pipeline cannot rely on any fixed template.

Coin

LIR2450

Li-ion

Non-ASCII dash in temperature range

Cylindrical

16340

Li-ion

Minimal spec, sparse PDF text

Pouch

PL-9059156

Li-polymer

Separate charge & discharge temps

Prismatic

CATL FFH3DS

$LiFePO_4$

Capacity in Ah, weight in kg

Special test — negative control

We gave the extractor the **project document** — not a datasheet. The extractor returned `null` for every field. It did not invent any battery specs. ✔ **perfect behaviour**

Level 1 — Schema & Extraction Pipeline

1 Units in the field name

`nominal_voltage_v, weight_g`

2 Closed vocabularies

Chemistry, FormFactor fixed list only

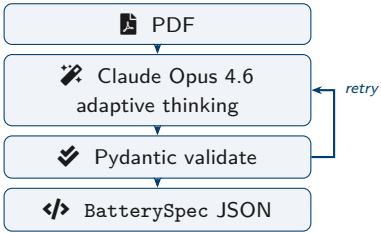
3 _raw field

Every value stores its source phrase

Hook that makes Level 3 possible

4 Strict mode `extra="forbid"`

LLM cannot add fields outside schema



Zero-shot (rules only) **vs Few-shot** (+ example)

Level 1 — Results

Zero-shot

97%

F_1

Precision	97.0%
Recall	97.0%
Hallucination	3.0%

rules only, no example

vs

Few-shot

95%

F_1

Precision	93.9%
Recall	96.9%
Hallucination	6.1%

rules + 1 example

Counterintuitive finding

Zero-shot wins. Our example leaked style the model added `Co Ltd` to every company name. **Clear rules beat a single example.**

Level 2 — Ontology Design

BEFORE plain literal

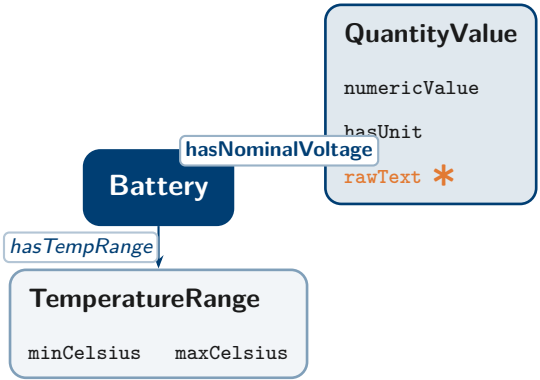
```
battery :voltage   3.6
```

AFTER every value is a node

```
battery :hasVoltage qv1
qv1 :numericValue  3.6
qv1 :hasUnit       Volt
qv1 :rawText       "3.6V"
```

rawText = *bridge to Level 3. Every value is traceable.*

Ontology authored in ***Protégé*** *as OWL 2.*



* **rawText** the provenance hook that unlocks Level 3

Level 2 — Knowledge Graph in Numbers

389 triples

4 batteries

27 QV nodes

7 temp ranges

rdflib

Turtle (.ttl)

SPARQL

Q6 voltage hierarchy check

```
SELECT ?model ?vCharge ?vNom ?vDis WHERE {
  ?b a bat:Battery ;
     bat:modelName ?model ;
     bat:hasChargeCutoffVoltage/bat:numericValue ?vCharge ;
     bat:hasNominalVoltage/bat:numericValue ?vNom ;
     bat:hasDischargeCutoffVoltage/bat:numericValue ?vDis .
  FILTER (?vCharge > ?vNom && ?vNom > ?vDis)
}
```

✔ Returns 4 rows invariant holds on every battery.

Level 3 — 8 Validation Rules

Structural

- **R5** Missing unit on a value
- **R6** Minimum $>$ Typical capacity

Physical

- **R1** Voltage hierarchy
- **R2** Li-ion $V_{\text{nom}} \in [3.0, 4.5]\,\text{V}$
- **R4** No negative scalars
- **R7** Charge $T \subset$ operating T
- **R8** Coin cell $< 30\,\text{g}$

Provenance

- **R3** Every `numericValue` must appear literally in `rawText`

Catches computed or hallucinated values.

Each rule = one SPARQL query. **Empty result = passes.** **Non-empty = violation.**

Level 3 — Results

Production KG

2

flags (Rule 3 only)

C-rate → Ampere conversions.
"23" absent from source text.

✔ **Expected behaviour surfaced for operator review.**

Corrupted KG

8/8

rules fired

Injected: broken voltage, −17 g, 250 g coin cell, 99 999 mAh.

✔ **Every error detected.**

The ontology catches what the LLM cannot catch itself.

Conclusion

97%
F_1
extraction

389
triples
knowledge graph

8/8
rules
all fire correctly

What we showed

- LLM + schema + ontology = viable pipeline
- Provenance makes hallucinations detectable
- All three levels delivered end-to-end

Honest limitations

- Rule 3 is conservative substring only
- Voltage rule is chemistry-coarse

The single most important decision: store the source phrase.

Cheap at extraction time. Trivial to carry through the KG. Turns a black box into an auditable pipeline.

Thank you.

Questions?